

JUNIO 2026

THE BRIDGE
BOOTCAMPS



INFORME DE HARDENING



Versión	Fecha	Estado	Clasificación
1.0	Junio 2026	Aprobado	Interno



Propósito y alcance

El hardening o bastionado consiste en reducir la superficie de ataque de un sistema eliminando o configurando de forma segura todo aquello que no es estrictamente necesario. Dicho de otro modo, se trata de dejar el sistema lo más “cerrado” posible antes de exponerlo al mundo.

Este informe aplica ese principio a Aupa!, revisando seis áreas clave: el sistema operativo del servidor, el servidor web y la API, la base de datos PostgreSQL, el contenedor Docker del módulo de Data Science, la configuración de red y la gestión de secretos.

Para cada área se explica qué debe hacerse y por qué, incluyendo los hallazgos concretos detectados en el repositorio que requieren corrección antes del despliegue en producción.



1 Sistema operativo

El servidor que aloja Aupa! debe mantenerse actualizado en todo momento. Aplicar parches de seguridad de forma regular es una de las medidas más sencillas y efectivas que existen. La mayoría de los ataques exitosos explotan vulnerabilidades conocidas para las que ya existe solución.

Cada servicio debe correr con su propio usuario del sistema, sin privilegios de administrador. El acceso SSH al servidor debe requerir clave pública (nunca contraseña) y el acceso directo como root debe estar desactivado. Cualquier servicio que no sea necesario para Aupa! (FTP, Telnet, escritorio remoto) debe eliminarse. Cada servicio activo es una puerta potencial que un atacante puede intentar abrir. El firewall solo debe tener abiertos los puertos 443 (HTTPS), 80 (para redirigir a HTTPS) y el SSH restringido a IPs conocidas.

2 Servidor web y API

Las cabeceras HTTP son metadatos que el servidor envía al navegador con cada respuesta. Configurarlas correctamente es gratuito y tiene un impacto directo en la seguridad del frontend de Aupa. Las más importantes son: Content-Security-Policy, que controla qué recursos puede cargar la página y evita ataques XSS; Strict-Transport-Security, que obliga al navegador a usar siempre HTTPS; y X-Frame-Options: DENY, que impide que la app sea embebida en iframes de terceros (clickjacking).

En el backend Express, la cabecera X-Powered-By: Express se envía por defecto y revela el framework utilizado debe desactivarse con `app.disable('x-powered-by')`. Los endpoints de login y registro deben tener un límite de peticiones por IP (rate limiting) para dificultar los ataques de fuerza bruta. La configuración de CORS debe permitir únicamente los orígenes propios de Aupa, nunca un comodín abierto.



5 Configuración de red

Todo el tráfico entre el navegador del usuario y Aupa debe ir cifrado mediante HTTPS. El puerto 80 (HTTP) debe existir únicamente para redirigir al usuario a HTTPS de forma automática, sin servir ningún contenido en texto plano. El protocolo TLS debe estar en su versión 1.2 como mínimo, siendo 1.3 el recomendado – las versiones antiguas tienen vulnerabilidades conocidas y deben desactivarse explícitamente.

En un despliegue en producción, la base de datos, la API y el servidor web deben estar en segmentos de red separados con reglas de firewall entre ellos. Para una aplicación pública como Aupa, añadir un CDN con WAF (Web Application Firewall) delante del servidor añade una capa de protección contra los ataques más comunes antes de que lleguen a la aplicación.

6 Gestión de secretos

Un secreto es cualquier dato sensible que permite acceder a un sistema: contraseñas, tokens, claves de API, el secreto de firma de los JWT. La regla fundamental es simple: los secretos nunca deben estar en el código fuente. Si alguien accede al repositorio – aunque sea de forma accidental, a través de un repositorio público o de un leak – tendría acceso inmediato a todos los sistemas.

El mecanismo correcto es usar variables de entorno: el código lee el valor de la variable en tiempo de ejecución, y el valor real solo existe en el servidor. El fichero `.env` local debe estar en `.gitignore` y nunca debe subirse al repositorio. En el caso de Aupa, los secretos del módulo de Data Science pueden configurarse directamente en el panel de variables de entorno de Render.

HALLAZGO CRÍTICO – AUTH.CONTROLLER.TS

El valor de `JWT_SECRET` se referencia directamente en el código. El JWT secret es la clave con la que se firman todos los tokens de sesión de Aupa: si un atacante lo conoce, puede fabricar tokens válidos y suplantar a cualquier usuario. Debe eliminarse del código y cargarse exclusivamente desde variables de entorno del servidor.



Como medida adicional, se recomienda configurar un hook de pre-commit con una herramienta como gitleaks o detect-secrets. Estos hooks analizan cada commit antes de que llegue al repositorio y bloquean cualquier fichero que contenga patrones de secretos. Es una red de seguridad que actúa en el momento justo (antes de que el daño sea irreversible) y que complementa la búsqueda de secretos que ya incorpora el script `daily_analysis.sh` del proyecto.

Por último, debe existir una política de rotación de secretos. Las contraseñas de la base de datos, el JWT secret y cualquier token de API externo deben renovarse periódicamente. Ante cualquier sospecha de que un secreto ha podido quedar expuesto, la rotación debe ser inmediata.

Los hallazgos detectados en el repositorio (la contraseña de base de datos en los logs, las credenciales hardcodeadas en el seed y el JWT secret en el controlador) son las correcciones más urgentes. El resto de medidas descritas en este informe conforman una base sólida de seguridad que, junto con el análisis OWASP y las reglas SIEM del proyecto, sitúa a Aupa en una postura de seguridad coherente con los estándares de la industria.

